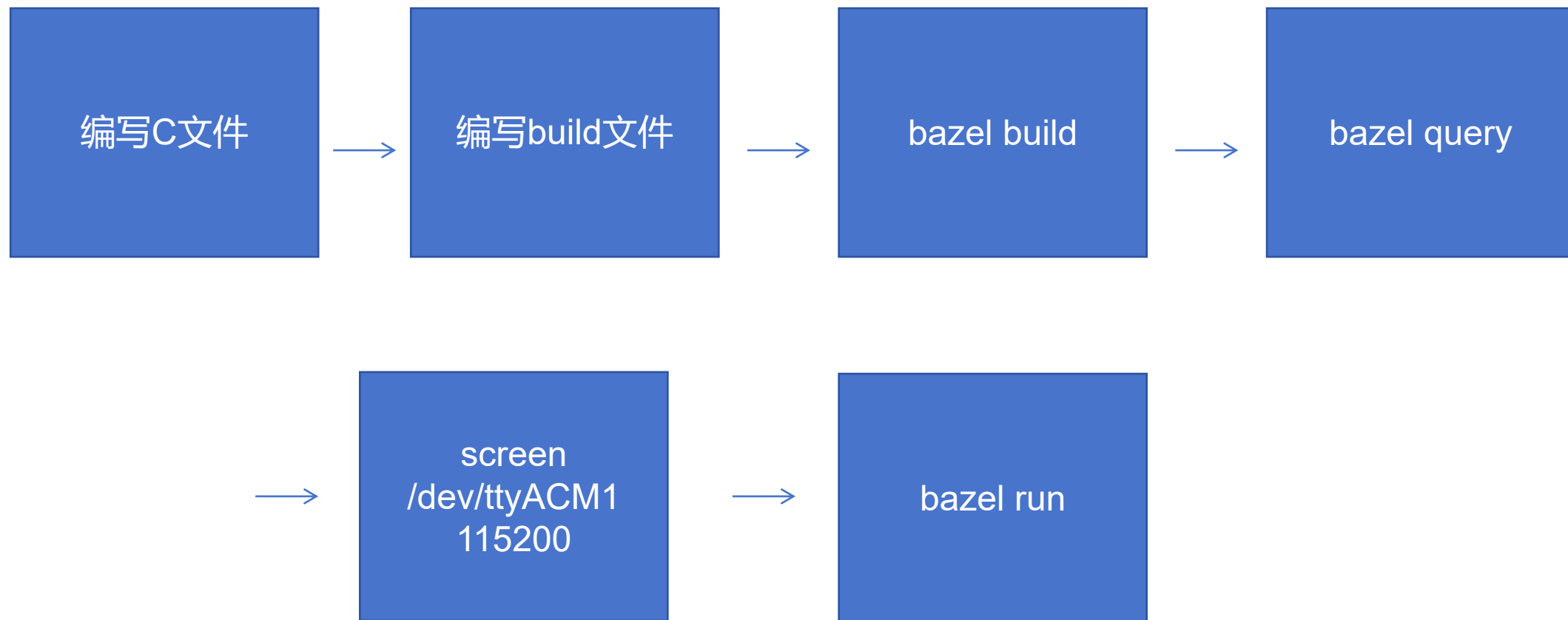


opentitan device软件编写测试

掌握运行自写测试的流程



测试.c的基本内容

```
#include "sw/device/my_tests/sm4/sm4_reg.h"  
#include "sw/device/lib/runtime/log.h"
```

包含的.h
依赖的其他库

```
#include "sw/device/lib/testing/test_framework/ottf_main.h"
```

```
#include "hw/top_earlgrey/sw/autogen/top_earlgrey.h"
```

```
OTTF_DEFINE_TEST_CONFIG();
```

默认结构

```
bool test_main(void) {  
    LOG_INFO(kSendData);  
  
    return true;  
}
```

BUILD文件的编写

```
load("//rules:opentitan.bzl", "OPENTITAN_CPU")
load(
    "//rules:opentitan_test.bzl",
    "OPENTITANTOOL_OPENOCD_DATA_DEPS",
    "OPENTITANTOOL_OPENOCD_TEST_CMDS",
    "ROM_BOOT_FAILURE_MSG",
    "cw310_params", //
    "dv_params",
    "opentitan_func_test",
    "verilator_params",
)
load("//rules/opentitan:defs.bzl", "EARLGREY_TEST_ENVS", "opentitan_test")
load("//rules:splice.bzl", "bitstream_splice")
load("//rules:otp.bzl", "STD_OTP_OVERLAYS", "otp_image", "otp_json", "otp_partition")

package(default_visibility = ["//visibility:public"])
exports_files(glob([
    "*.c",
    "*.h",
]))
```

这是通用部分

测试名为puf_smoketest,同名.c文件
依赖一个自己的库, 下面定义自己的库, 自己的库函数写在puf.c中, .h中定义常量、宏、声明函数。需要用其他的库函数, 要先包含.h进来。xxx_smoketest.c和xxx.c中
include xxx.h

在测试.c的同层写一个名为BUILD的文件

```
opentitan_func_test(
    name = "puf_smoketest",
    srcs = ["puf_smoketest.c"],
    targets = ["cw310_test_rom",
               "verilator"],
    deps = [

        ":puf_lib",

    ],
)
cc_library(
    name = "puf_lib",
    srcs = ["puf.c"],
    hdrs = ["puf.h"],
    target_compatible_with = [OPENTITAN_CPU],
    deps = [
        "//hw/top_earlgrey/sw/autogen:top_earlgrey",
        "//sw/device/lib/runtime:log",
        "//sw/device/lib/testing/test_framework:ottf_main",
        "//sw/device/my_tests/base:base_lib",
        "//sw/device/my_tests/rs_encode:rs_encode_lib",
        "//sw/device/my_tests/rs_decode:rs_decode_lib",
    ],
)
```

构建测试，编译

```
bazel build //sw/device/examples/hello_world:my_hello_world
```

若无法build，提示gcc的问题，则升级gcc到11

查看有哪些测试可以用，目的是为了找到bin文件

```
bazel query 'attr(tags, cw310, tests(//sw/device/tests/...))'
```

用verilator测试并看波形

只仿真不记录波形速度会快很多

```
bazel test //sw/device/my_tests/sm4:sm4_smoketest_sim_verilator \
--test_output=streamed \
--test_timeout=2000 \
--test_arg=--verilator-args=--trace=/tmp/sim.fst
```

终端上打印log

```
gtkwave /tmp/sim.fst
```

在**选好信号**之后在file 中write save file as，可以将波形另存为一个gtkw文件。重新仿真生成.fst后，打开fst文件，在file中打开保存的gtkw文件，就会自动将**选好的信号**拉出来。

将终端log打印到文本： `script log.txt` 将会在当前路径下生成一个txt文件记录产生的log。 `exit`终止

可在`bazel query 'filter(".*_fpga_cw310", kind(".", //sw/device/tests/...))'` 的log中查找哪些可用测试文件

上板测试

查看有哪些测试文件

```
bazel query 'filter(".*_fpga_cw310", kind(".", //sw/device/my_tests/...))'
```

```
screen /dev/ttyACM1 230400,cs8,-ixon,-ixoff 开串口
```

注意：波特率取决于soc根时钟

`ctrl+A+D` 分离screen

`screen -ls` 查看有哪些后台screen

`screen -r 后台ID` 恢复后台的screen

`screen -XS ID quit` 关闭后台运行的screen (多个可选)

`screen -X quit` 关闭单个

```
bazel run //sw/host/opentitantool bootstrap $(ci/scripts/target-location.sh  
//sw/device/examples/hello_world:my_hello_world_fpga_cw310_bin)
```

加载测试软件到SOC的flash

一些编程的tips

- 1、opentitan里边用abs_mmio_write32写入数据
src/sw/device/lib/base/abs_mmio.h

关于编译后总线未能按次序去读写寄存器（如何防止自动优化掉一些步骤）



```
asm volatile("" ::: "memory");
```



在C和C++中，`asm volatile("" ::: "memory");` 是一个内嵌汇编语句，用于告诉编译器不要优化该代码段。

让我们分解这个语句：

1. `asm`：这是内嵌汇编的关键字，允许你在C或C++代码中插入汇编代码。
2. `volatile`：这告诉编译器不要优化这个汇编代码段。即使编译器通常会尝试优化代码以提高性能，`volatile` 关键字会阻止它这样做。
3. `""`：这是汇编操作的空字符串。实际上，这里没有具体的汇编操作，只是一个占位符。
4. `::: "memory"`：这告诉编译器这个汇编代码段可能会影响到内存，因此不要重新排序任何内存访问以改善性能。

这个语句通常用于确保编译器不会重新排序某些关键的内存访问或保证某些操作的顺序。例如，当你在多线程环境中编写代码时，你可能需要确保某些内存操作不会被优化或重新排序，以确保线程安全。

但要注意，过度使用内嵌汇编和 `volatile` 可能会导致代码难以理解和维护，并且可能会降低性能。因此，除非绝对必要，否则应避免使用它。

由文心大模型 3.5 生成

<https://blog.csdn.net/sxudong2010/article/details/118616418>

https://blog.csdn.net/qq_36412526/article/details/84640924

关于C中用指针操作一段连续的存储器映射



指针的自增 (Incrementing Pointer)

如果将指针增加1，指针将开始指向下一个位置。这与一般的算术运算有些不同，因为指针的值将增加指针所指向的数据类型的大小。

我们可以通过在循环中使用指针的自增操作来遍历数组，使指针依次指向数组的每个元素，对其执行一些操作，并在循环中更新指针。

指针自增的规则如下：

新地址 = 当前地址 + i * 数据类型的大小

其中， i 是指针增加的数量。

```
data_in_addr = (unsigned int *) (RS_ENCODE_DATA_IN_41_REG_ADDR - (i+1) * (0x1u));
```

```
temp1 = (unsigned int)(data_in_addr) - 0x4; //一种操作指针的方法
```

直接操作指针

间接操作指针

指针不能按一般变量去运算。

```
unsigned int *data_in_addr=(unsigned int *) (TOP_EARLGREY_RS_ENCODE_BASE_ADDR+RS_ENCODE_DATA_IN_41_REG_OFFSET);
unsigned int *data_out_addr=(unsigned int *) (TOP_EARLGREY_RS_ENCODE_BASE_ADDR+RS_ENCODE_ENCODED_DATA_OUT_49_REG_OFFSET);
unsigned int temp1 = (unsigned int) data_in_addr;
//LOG_INFO("temp1=%08x",temp1);
unsigned int temp2 = (unsigned int) data_out_addr;
//LOG_INFO("temp2=%08x",temp2);

for (uint8_t i = 0; i < 42; i++)
{
    data_in_addr=(unsigned int *)temp1;
    *data_in_addr = data_input[i];
    //LOG_INFO("i=%08x",i);
    LOG_INFO("data_in_addr=%08x",data_in_addr);
    temp1 = (unsigned int)(data_in_addr) - 0x4; //一种操作指针的方法
}
```

一种操作指针写入存储器（广义）的方法

opentitan 安全启动

目标:

- 1、在opentitan device (sim or FPGA)上运行安全启动流程: ROM——ROM_EXT——BL0
- 2、总结opentitan脱机运行模式(上电直接安全启动、直到用户代码)的开发流程

安全启动相关资料

https://opentitan.org/book/sw/device/silicon_creator/index.html rom和rom_ext软件介绍

https://opentitan.org/book/doc/security/specs/secure_boot/index.html 安全启动具体内容

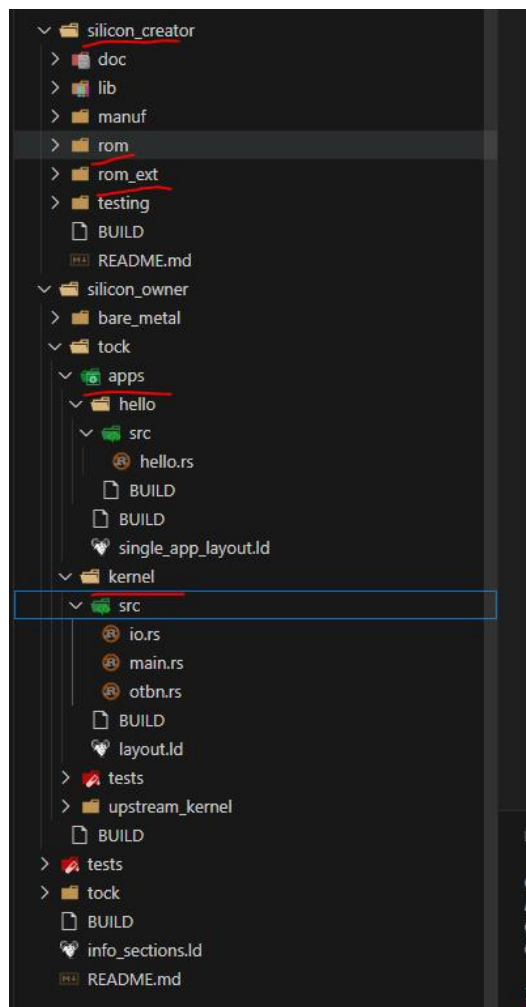
[sw/device/silicon_creator/rom/data/rom_e2e_testplan.hjson](#) ROM testplan

[sw/device/silicon_creator/rom_ext/data/rom_ext_e2e_testplan.hjson](#) ROM_EXT testplan

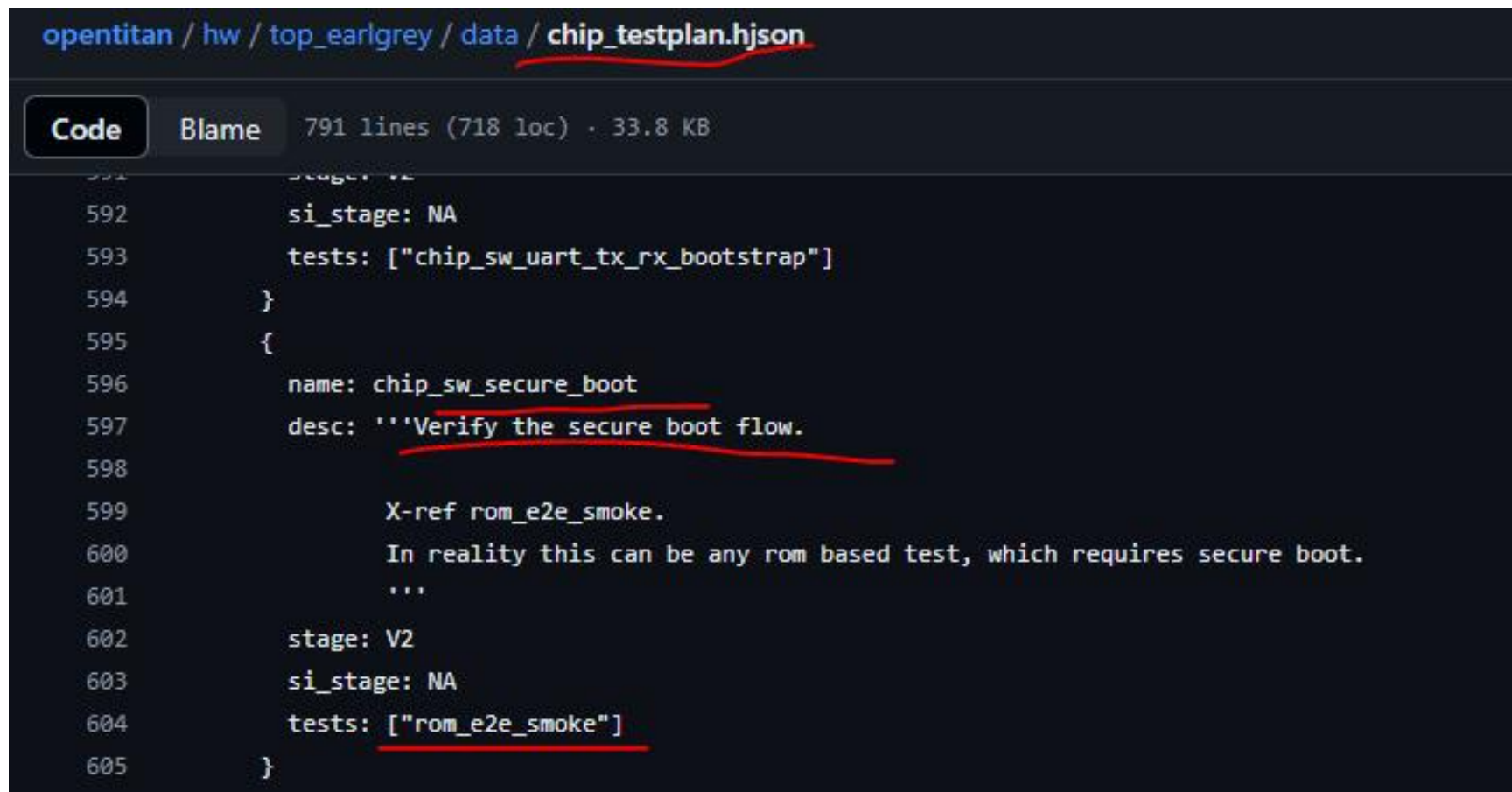
<https://lowrisc.zulipchat.com/#narrow/stream/213499-OpenTitan/topic/secure.20boot> 问题讨论

<https://opentitan.org/book/rules/opentitan/index.html#execution-environments> 测试的执行环境

方法一：rom_e2e_smoke_sim_verilator



相关代码



chip级testplan

```
BUILD  rom_e2e_testplan.hjson  rom_manual_testplan.hjson
silicon_creator > rom > data > rom_e2e_testplan.hjson

1 // Copyright lowRISC contributors.
2 // Licensed under the Apache License, Version 2.0, see LICENSE for details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 {
6   name: "rom_e2e"
7
8   testpoints: [
9     {
10      name: rom_e2e_smoke
11      desc: '''Verify that ROM can boot a ROM_EXT with default infrastructure configurations.
12
13         - The valid ROM_EXT should be a test program that returns `true`, optimizing for speed.
14         - The test program should be launched via the OTTF.
15         '''
16      tags: ["rom", "verilator", "dv", "fpga", "silicon"]
17      stage: V2
18      tests: ["rom_e2e_smoke"]
19    }
20  ]
}
```



```

//sw/device/silicon_creator/rom/e2e/weak_straps:e2e_weak_straps_9_sim_verilator
//sw/device/silicon_creator/rom/e2e/weak_straps:sw_strap_value_sim_verilator
//sw/device/silicon_creator/rom/keys/fake/rsa:rsa_unittest
//sw/device/silicon_creator/rom/keys/fake/spx:spx_unittest
//sw/device/silicon_creator/rom/keys/real/rsa:rsa_unittest
//sw/device/silicon_creator/rom/keys/real/spx:spx_unittest
Loading: 0 packages loaded
houdonghui@secure-Precision-3660:~/opentitan$ bazel test //sw/device/silicon_creator/rom/e2e:rom_e2e_smoke_sim_verilator \
> --test_output=streamed \
> --test_arg=--verilator-args=--trace=/tmp/sim.fst
WARNING: Streamed test output requested. All tests will be run locally, without sharding, one at a time
INFO: Analyzed target //sw/device/silicon_creator/rom/e2e:rom_e2e_smoke_sim_verilator (589 packages loaded, 39345 targets configured).
INFO: Found 1 test target...
INFO: From Action hw/top_earlgrey/alert_handler_regs.h:
/home/houdonghui/opentitan/util/reggen/gen_chheader.py:215: UserWarning: Cannot generate a module define of type logic [NAlerts-1:0][NLpgWidth-1:0]
  warnings.warn("Cannot generate a module define of type {}")
/home/houdonghui/opentitan/util/reggen/gen_chheader.py:215: UserWarning: Cannot generate a module define of type logic [NAlerts-1:0]
  warnings.warn("Cannot generate a module define of type {}")
[1,551 / 1,553] FuseSoC hw/build.verilator_real; 26s linux-sandbox

```

在verilator上执行ROM测试，时间很久.....

方法二：在ROM_EXT阶段执行某个测试【带安全启动】

OpenTitan > secure boot  TODAY

 **Mr. Jack**    3:29 PM

Hello, I would like to run a complete secure boot process on OpenTitan, which is described in the document as a secure boot from ROM to ROM-EXT and then to BL0. What do I need to do? I am currently using testrom for testing on both Verilator and FPGA. I would like to test the complete secure boot time of OpenTitan. How do I need to develop it?

 **Douglas Reis**  4:27 PM

Hello Mr. Jack,
If you are using the testrom I'm assuming that you are running the bazel command such as:

```
./bazelisk.sh test //sw/device/tests:aes_smoketest_fpga_cw310_test_rom
```

To run the same test with the ROM you should do:

```
./bazelisk.sh test //sw/device/tests:aes_smoketest_fpga_cw310_rom_with_fake_keys
```

To run with the ROM + ROM_EXT you should do:

```
./bazelisk.sh test //sw/device/tests:aes_smoketest_fpga_cw310_sival_rom_ext
```

 **Mr. Jack** 5:46 PM

Thank you for your answer. What do I need to do if I want to run the ROM+ROM-EXT test on Verilator?

@Douglas Reis For example, if I want to execute an aes_smoketest with secure boot on a verifier or a test I wrote myself, what parameters do I need to specify in the BUILD file? And what kind of testing commands need to be executed?

 **Douglas Reis**  EDITED 6:20 PM

You have to add the desired execution environment to your bazel target:

```
exec_env = {  
    "//hw/top_earlgrey:fpga_cw310_test_rom": None,  
    "//hw/top_earlgrey:fpga_cw310_rom_with_fake_keys": None,  
    "//hw/top_earlgrey:sim_verilator_rom_with_fake_keys": None,  
},
```

Then you run with the bazel command above, using the `exec_env` as a suffix to select which platform do you want to run.

关于软件执行环境的概念

Execution Environments

A core concept of the new rule structure is that of an execution environment (or **exec_env**). An execution environment is a unique hardware and software configuration. Our `exec_envs` include the `sim_verilator`, `sim_dv` and `fpga_cw310` environments (among others). In addition to simply referring to the simulation or emulation platforms, the `exec_envs` also include the ROM, OTP configuration and linker maps for a given environment. For example, the `fpga_cw310_test_rom` environment is used to execute test programs on the CW310 board with the test program linked in flash memory at the ROM_EXT stage and executed after the `test_rom`. There may be several additional environments for the `fpga_cw310` that execute the test program under different conditions (ie: under the real ROM or under the ROM_EXT).

An `exec_env` defines everything about building and dispatching testing binaries for that environment. In addition to the resources needed to build and dispatch binaries (e.g. the `test_rom`, linker scripts, signing keys, etc), the `exec_env` also supplies some starlark functions to help the various environments perform the build and test actions.

- Building
 - An `exec_env`-specific provider, such as `Cw310BinaryInfo` or `SimVerilatorBinaryInfo`.
 - An `exec_env`-specific `transform` function that converts binaries into the environment's required form (e.g. `vmem` files for DV or Verilator).
- Testing
 - An `exec_env`-specific `test_dispatch` function that configures the test environment and dispatches the binary to that environment.

The `exec_envs` are provided to the `opentitan_binary` and `opentitan_test` rules in order to allow them to perform their function.

<https://opentitan.org/book/rules/opentitan/index.html#execution-environments>

```
//sw/device/tests:aes_entropy_test_fpga_cw310_rom_with_fake_keys
//sw/device/tests:aes_entropy_test_fpga_cw310_sival
//sw/device/tests:aes_entropy_test_fpga_cw310_sival_rom_ext
//sw/device/tests:aes_entropy_test_fpga_cw310_test_rom
//sw/device/tests:aes_entropy_test_sim_dv
//sw/device/tests:aes_entropy_test_sim_verilator
```

```
opentitan_test(  
  name = "aes_entropy_test",  
  srcs = ["aes_entropy_test.c"],  
  exec_env = {  
    "//hw/top_earlgrey:fpga_cw310_test_rom": None,  
    "//hw/top_earlgrey:fpga_cw310_rom_with_fake_keys": None,  
    "//hw/top_earlgrey:fpga_cw310_sival": None,  
    "//hw/top_earlgrey:fpga_cw310_sival_rom_ext": None,  
    "//hw/top_earlgrey:sim_dv": None,  
    "//hw/top_earlgrey:sim_verilator": None,  
  },  
  deps = [  
    "//hw/ip/aes:model",  
    "//hw/top_earlgrey/sw/autogen:top_earlgrey",  
    "//sw/device/lib/base:memory",  
    "//sw/device/lib/base:mmio",  
    "//sw/device/lib/dif:aes",  
    "//sw/device/lib/runtime:log",  
    "//sw/device/lib/testing:aes_testutils",  
    "//sw/device/lib/testing:entropy_testutils",  
    "//sw/device/lib/testing/test_framework:ottf_main",  
  ],  
)
```

重新总结开发流程【带安全启动】